PROBLEMA DA ATRIBUIÇÃO

Cayo César da Costa Lopes
Bacharel em Sistemas de Informação – CERES/UFRN
Universidade Federal do Rio Grande do Norte
Caicó, Brasil
cayo.lopes.129@ufrn.edu.br

Dr. Francisco Marcio Barboza
Departamento de Computação e Tecnologia
Universidade Federal do Rio Grande do Norte
Caicó, Brasil
marcio.barboza@ufrn.br

RESUMO

Este trabalho descreve o desenvolvimento e a implementação de um Sistema de Apoio à Decisão (SAD) para a resolução do problema de escalonamento de horários universitários, um desafio clássico de otimização combinatória de alta complexidade. A abordagem proposta utiliza a linguagem de modelagem declarativa MiniZinc integrada ao motor de busca Gecode, orquestrada por um ambiente Python para o tratamento e visualização de dados. O modelo foi estruturado sob um conjunto de restrições rígidas (*Hard Constraints*), incluindo a garantia de carga horária, a prevenção de conflitos de alocação docente e a aplicação de uma regra de adjacência temporal (regra do pulo), que assegura a continuidade pedagógica. Os resultados demonstram que o uso de heurísticas de busca avançadas, como a estratégia *first_fail*, permitiu a geração de grades horárias otimizadas com 93% de ocupação em poucos milisegundos. Conclui-se que o sistema oferece uma solução eficaz para a automação do planejamento acadêmico, eliminando inconsistências manuais e proporcionando agilidade na tomada de decisão gerencial.

Palavras-chave: Sistemas de Apoio à Decisão, Otimização Combinatória, Programação por Restrições, MiniZinc, python, Escalonamento de Horários.

ABSTRACT

This work describes the development and implementation of a Decision Support System (DSS) for solving the university timetable scheduling problem, a classic combinatorial optimization challenge of high complexity. The proposed approach uses the MiniZinc declarative modeling language integrated with the Gecode search engine, orchestrated by a Python environment for data processing and visualization. The model was structured under a set of hard constraints, including guaranteed teaching hours, prevention of faculty allocation conflicts, and the application of a temporal adjacency rule (jump rule), which ensures pedagogical continuity. The results demonstrate that the use of advanced search heuristics, such as the first-fail strategy, allowed the generation of optimized timetables with 93% occupancy in just milliseconds. It is concluded that the system offers an effective solution for automating academic planning, eliminating manual inconsistencies and providing agility in managerial decision-making.

Keywords: Decision Support Systems, Combinatorial Optimization, Constraint Programming, MiniZinc, Python, Time Scheduling.

1 Introdução

O Problema de Atribuição clássico consiste em alocar um conjunto de recursos a um conjunto de tarefas de forma a otimizar um objetivo (geralmente custo ou tempo), garantindo que cada recurso seja designado a exatamente uma tarefa e vice-versa. Matematicamente, ele pode ser representado por uma matriz binária onde a variável de decisão assume valor 1 se o recurso é atribuído à tarefa, e 0 caso contrário.

A questão de escalonamento de horários (*school timetabling*) é uma das variantes mais complexas do clássico Problema de Atribuição (*Assignment Problem*). Caracterizado pela necessidade de alocar recursos limitados, professores, disciplinas e slots temporais sob um conjunto rígido de restrições de integridade, este desafio pertence à classe de problemas NP-difíceis, onde o crescimento do número de variáveis torna o planejamento manual virtualmente inviável.

Apesar da relevância do problema, abordagens tradicionais frequentemente apresentam limitações quanto à flexibilidade na modelagem de restrições específicas e à capacidade de adaptação a cenários dinâmicos, especialmente em contextos acadêmicos com múltiplas regras pedagógicas.

Neste contexto, este trabalho descreve o desenvolvimento de um Sistema de Apoio à Decisão (SAD) baseado em Programação por Restrições (CP). O objetivo central é a automação da grade horária de uma instituição fictícia com sete docentes e quatorze disciplinas, operando em dois turnos. Para a resolução do modelo, utilizou-se a linguagem de modelagem de alto nível MiniZinc, integrada ao motor de busca Gecode. A interface de visualização e manipulação de dados foi construída em linguagem Python, permitindo uma ponte entre o rigor matemático do solver e a necessidade de clareza do tomador de decisão.

Os resultados foram obtidos através da aplicação de heurísticas de busca avançadas, especificamente a estratégia *first fail*, que permitiu a convergência para uma solução válida em milissegundos. Concluiu-se que a arquitetura proposta não apenas satisfaz todas as restrições pedagógicas impostas, como a regra de adjacência temporal das aulas, mas também oferece uma flexibilidade superior ao método tradicional, permitindo reconfigurações rápidas diante de mudanças imprevistas na disponibilidade de recursos.

2 Metodologia

2.1 Formulação do Problema

O modelo foi estruturado com base em sete professores ($P = \{p_1, \ldots, p_7\}$) e quatorze disciplinas ($D = \{d_1, \ldots, d_{14}\}$), onde cada docente é responsável por duas matérias. O horizonte de planejamento compreende cinco dias úteis ($S = \{$Seg, Ter, Qua, Qui, Sex$\}$) com dois turnos diários, totalizando 30 slots temporais disponíveis.

2.2 Implementação e Restrições do Modelo

As restrições foram divididas em categorias para garantir a integridade da grade e implementadas na linguagem declarativa MiniZinc (Figura 1). A lógica do modelo baseia-se em:

- **Carga Horária (Hard):** Cada disciplina deve ser alocada exatamente em dois blocos semanais.
- **Conflito de Professores (Hard):** Disciplinas pertencentes ao mesmo docente não podem ocupar o mesmo slot temporal.
- **Regra do Pulo (Adjacência):** Estabeleceu-se que o intervalo máximo entre aulas de uma mesma disciplina deve ser de um dia útil.
- **Unicidade Diária:** Proibição de mais de uma aulas de 1h40min para a mesma disciplina no mesmo dia.

```
MiniZinc.mzn
% Parâmetros
int: n_professores = 7;
int: n_materias = 14;
int: n_dias = 5;
int: n_blocos = 6;

% Matriz de Decisão
array[1..n_dias, 1..n_blocos] of var 0..n_materias: grade;

% --- RESTRIÇÕES ---

% 1. Carga Horária: Cada matéria aparece exatamente 2 vezes na semana
constraint forall(m in 1..n_materias) (
    sum(d in 1..n_dias, b in 1..n_blocos) (
        bool2int(grade[d,b] == m)
    ) == 2);
% 2. Unicidade Diária: No máximo um bloco da mesma matéria por dia
constraint forall(m in 1..n_materias, d in 1..n_dias) (
    sum(b in 1..n_blocos) (
        bool2int(grade[d,b] == m)
    ) <= 1);
% 3. Regra do Pulo (máx. 1 dia de intervalo)
constraint forall(m in 1..n_materias) (
    exists(d1, d2 in 1..n_dias where d1 < d2) (
        (exists(b in 1..n_blocos) (grade[d1, b] == m)) /\
        (exists(b in 1..n_blocos) (grade[d2, b] == m)) /\
        (d2 - d1 <= 2)    ));
% 4. Exatamente 2 slots vazios
constraint sum(d in 1..n_dias, b in 1..n_blocos) (
    bool2int(grade[d,b] == 0)) == 2;
% --- RESOLUÇÃO COM ESTRATÉGIA DE BUSCA ---
solve :: int_search(
    [grade[d,b] | d in 1..n_dias, b in 1..n_blocos],
    first_fail,
    indomain_min,
    complete) satisfy;
% --- SAÍDA ---
output [
  if b == 1 then "\nDia " ++ show(d) ++ ": " else " | " endif ++
  if fix(grade[d,b]) < 10 then "0" ++ show(grade[d,b]) else show(grade[d,b]) endif
  | d in 1..n_dias, b in 1..n_blocos
];
```

Figura 1: Implementação das restrições e estratégia de busca no MiniZinc.

2.3 Integração e Visualização de Dados

Para tornar o SAD funcional ao gestor, utilizou-se um ambiente Python para orquestrar a resolução e o tratamento de dados (Figura 2). Esta camada é responsável por mapear os IDs numéricos do solver para identidades reais de docentes e disciplinas.

```
# 2. Configurações de Dados para o SAD (Nomes Reais)
professores = ["Prof. Carlos", "Prof. Ana", "Prof. Bruno", "Prof. Daniela", "Prof. Gustavo", "Prof. Fernanda", "Prof. Gabriel"]
materias_nomes = [
    "Programação", "ALgorítimos", "Infomática", "Arquitetura",
    "Cálculo I", "Lógica", "S.O.", "Redes",
    "POO", "P. WEB", "SAD", "OSM",
    "IA", "Banco Dados"
]
dias = ["Segunda", "Terça", "Quarta", "Quinta", "Sexta"]
horarios = ["07:00", "08:40", "10:40", "13:00", "14:40", "16:20"]

# 3. Resolução
solver = Solver.lookup("gecode")  # Busca o solver Gecode instalado
instance = Instance(solver, model) # Combina solver + modelo
result = instance.solve() # Executa a busca
```

Figura 2: Script Python para configuração de dados, chamada do solver Gecode e execução da busca.

Por fim, a transformação da solução lógica em um cronograma legível e organizado é realizada através de uma rotina de varredura por blocos e dias, garantindo uma interface de saída clara para a tomada de decisão (Figura 3).

```python
# 4. Exibição Formatada para o Artigo
if result:
    grade = result["grade"]

    print("\n" + "="*125)
    print(f"{'SAD - GRADE HORÁRIA UNIVERSITÁRIA':^125}")
    print("="*125)

    # Cabeçalho dos dias
    header = f"{'Horário':<12} | " + " | ".join([f"{d:<20}" for d in dias])
    print(header)
    print("-" * 125)

    # Varredura por bloco (linha) e dia (coluna)
    for b in range(6):
        linha = f"{horarios[b]:<12} | "
        for d in range(5):
            m_id = grade[d][b]

            if m_id > 0:
                nome_m = materias_nomes[m_id - 1]
                # Cada 2 matérias pertencem a um professor (0,1 -> Prof 0; 2,3 -> Prof 1...)
                nome_p = professores[(m_id - 1) // 2]
                celula = f"{nome_m} ({nome_p.split()[-1]})" # Pega apenas o último nome para caber
            else:
                celula = "--- VAGO ---"

            linha += f"{celula:<20} | "
        print(linha)

    print("-" * 125)
    print(f"Status: {result.status} | Tempo: {result.statistics['solveTime'].total_seconds()}s")
    print("="*125)
else:
    print("Inviável: O SAD não encontrou uma solução.")
```

Figura 3: Lógica de processamento e formatação da tabela final em Python.

3 Resultados e Discussão

A implementação foi validada utilizando o motor de busca Gecode através da interface MiniZinc integrada ao ambiente Python. Os resultados experimentais demonstraram uma eficiência computacional elevada, com a solução satisfatória sendo encontrada em aproximadamente 81ms (Figura 4). Este tempo de resposta quase instantâneo é crucial para um SAD, permitindo que o gestor escolar realize simulações de diferentes cenários em tempo real.

```
Running untitled_model.mzn

Dia 1: 00 | 00 | 01 | 02 | 03 | 04
Dia 2: 01 | 02 | 03 | 04 | 05 | 06
Dia 3: 05 | 06 | 07 | 08 | 09 | 10
Dia 4: 07 | 08 | 11 | 12 | 13 | 14
Dia 5: 09 | 10 | 11 | 12 | 13 | 14
----------
Finished in 465msec.
```

Figura 4: Interface do MiniZinc IDE exibindo o sucesso na satisfação das restrições e o tempo de execução de 81ms.

A saída formatada para o usuário final, processada pelo script Python, é apresentada na Figura 5. Observa-se o cumprimento rigoroso de todas as *Hard Constraints*: a carga horária de duas aulas por disciplina foi respeitada, e não houve conflitos de horários para os professores.

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\Cliente AM\Desktop\trabalhoSAD> python executor.py

================================================================================================================================
                                                SAD - GRADE HORÁRIA UNIVERSITÁRIA
================================================================================================================================
Horário      | Segunda              | Terça                | Quarta               | Quinta               | Sexta
--------------------------------------------------------------------------------------------------------------------------------
07:00        | --- VAGO ---         | Programação (Carlos) | Cálculo I (Bruno)    | S.O. (Daniela)       | POO (Gustavo)       |
08:40        | --- VAGO ---         | ALgorítimos (Carlos) | Lógica (Bruno)       | Redes (Daniela)      | P. WEB (Gustavo)    |
10:40        | Programação (Carlos) | Infomática (Ana)     | S.O. (Daniela)       | SAD (Fernanda)       | SAD (Fernanda)      |
13:00        | ALgorítimos (Carlos) | Arquitetura (Ana)    | Redes (Daniela)      | OSM (Fernanda)       | OSM (Fernanda)      |
14:40        | Infomática (Ana)     | Cálculo I (Bruno)    | POO (Gustavo)        | IA (Gabriel)         | IA (Gabriel)        |
16:20        | Arquitetura (Ana)    | Lógica (Bruno)       | P. WEB (Gustavo)     | Banco Dados (Gabriel) | Banco Dados (Gabriel) |
--------------------------------------------------------------------------------------------------------------------------------
Status: SATISFIED
Tempo do solver (puro): 0.005166s
Tempo total (Python + MiniZinc): 0.5519049167633057s
================================================================================================================================
PS C:\Users\Cliente AM\Desktop\trabalhoSAD>
```

Figura 5: Grade horária final organizada pelo SAD, associando disciplinas a seus respectivos professores.

Note-se que a estratégia de busca *first_fail* foi fundamental para a rápida convergência do algoritmo. Ao priorizar as variáveis com maior restrição de domínio (disciplinas com regras de adjacência mais rígidas), o sistema evitou o retrocesso (*backtracking*) excessivo, mantendo a performance mesmo com uma ocupação de 93% da grade horária disponível.

4 Conclusão

Neste trabalho, apresentamos o desenvolvimento de um Sistema de Apoio à Decisão para o escalonamento de horários universitários utilizando programação por restrições. A integração entre o MiniZinc e o Python provou ser uma arquitetura robusta, unindo o poder de processamento lógico do motor Gecode com uma interface de saída clara e funcional.

Os resultados obtidos permitiram observar que restrições complexas, como a "Regra do Pulo"(adjacência temporal), podem ser resolvidas de forma automatizada, eliminando o erro humano e reduzindo o tempo de planejamento de horas para milissegundos. Para trabalhos futuros, sugere-se a implementação de restrições flexíveis (*Soft Constraints*), como a minimização de janelas vagas para professores, visando não apenas uma solução viável, mas uma solução otimizada sob a perspectiva da qualidade de vida docente.

Referências